ĐẶC TẢ THIẾT KẾ gemm_systolic_array

Module của `gemm_accel_top` · instance `u_array`

Mục lục

1 Tổng quan **2**

1.1 Tính năng chính . . . 2

1.2 Tham số . . . 2

1.3 Tín hiệu giao tiếp . . . 3

2 Đặc tả thanh ghi **5**

3 Mô tả chức năng **6**

3.1 Tổ chức chức năng . . . 6

3.2 Module con . . . 7

3.3 Timing . . . 13

3.4 Reset và soft clear . . . 15

3.5 Quy tắc hiện thực . . . 15

1 Tổng quan

1.1 Tính năng chính

Sau một xung `i_pe_start`, array tự duyệt mọi tile 8×8 của C, tự đọc A và B từ scratchpad, tính tích lũy INT32 trong 64 PE, rồi xả kết quả sang VPU theo từng hàng, mỗi chu kỳ một hàng. Khi hàng cuối cùng đã đi qua VPU và được ghi vào buffer C, array phát một xung `o_pe_done`. Array không bao giờ chờ ai và không có đầu ra lỗi.

Chức năng

- Với cấu hình M, N, K của `i_matrix_dim`, array xuất đúng một lần mỗi phần tử $C[i][j] = \sum_k A[i][k] \cdot B[k][j]$, 0 ≤ i < M, 0 ≤ j < N, ở dạng INT32 (tích 16 bit có dấu, cộng dồn wrap mod 2^{32})
- Kết quả được xuất theo đoạn hàng: một lần xuất gồm hàng i, cột $j_0 \ldots j_0 + 7$ và mask 8 bit; lane có mask = 0 mang giá trị không xác định
- Kết quả không phụ thuộc `L_VPU`
- Array chỉ đọc buffer A, B và chỉ xuất sang VPU; không đọc buffer C, không truy cập DRAM, không truy cập buffer BIAS
- Array không có đầu ra lỗi. Array giả định M, N, K đã hợp lệ (Main FSM đã kiểm tra descriptor trước khi phát `i_pe_start`)
- Dataflow **Output-Stationary**: PE(r, c) giữ `acc` của $C[i_0 + r][j_0 + c]$ qua toàn bộ k = 0 … K−1

> **QUYẾT ĐỊNH BASELINE — D-04**
> P = 8 cố định, 64 PE. Các đại lượng dẫn xuất cũng cố định: Q = P/4 = 2, LP = $\log_2 P$ = 3, `AB_VEC_WIDTH` = 8·P = 64 bit, `ROW_VEC_WIDTH` = 32·P = 256 bit. Tài liệu này không mô tả biến thể P khác; mọi công thức viết theo P chỉ để đọc cho rõ nghĩa, giá trị luôn là 8.

1.1.1 Thuật toán

C được chia thành lưới $T_M \times T_N$ tile, $T_M = \lceil M/8 \rceil$, $T_N = \lceil N/8 \rceil$; số tile $N_T = T_M T_N$. Tile thứ n có (t_i, t_j) với $n = t_i T_N + t_j$ (t_j **chạy trong,** t_i **chạy ngoài**), phủ hàng $i_0 = 8t_i \ldots i_0 + 7$ và cột $j_0 = 8t_j \ldots j_0 + 7$. Trong tile, PE(r, c) tích lũy $C[i_0 + r][j_0 + c]$ qua toàn bộ k = 0 … K−1. Chi tiết lý thuyết ở tài liệu thuật toán của dự án (`docs/algorithm/gemm_systolic_vpu_algorithm.pdf`, mục 5 “Systolic array Output-Stationary”).

1.2 Tham số

Bảng 1.1: Tham số

Tham số	Giá trị	Ý nghĩa
`P`	8	Kích thước array, cố định. Nối từ `PE_ARRAY_SIZE` của top
`L_VPU`	4	Độ trễ của VPU (chu kỳ), nối từ `gemm_pkg::VPU_LATENCY` = 3 + `MUL_STAGES` = 4
`IDX_W`	5	localparam: độ rộng chỉ số hàng/cột/k (0 … 31)
`DIM_W`	6	localparam: độ rộng giá trị M, N, K (1 … 32)

Bảng 1.2: Hằng dẫn xuất từ P (không phải tham số)

Hằng	Giá trị	Công thức, nơi dùng
`AB_VEC_WIDTH`	64 bit	8·P: độ rộng vector A, B (`i_a_rd_data`, `i_b_rd_data`)
`ROW_VEC_WIDTH`	256 bit	32·P: độ rộng đoạn hàng kết quả (`o_row_data`)
Q	2	P/4: số sub-bank của buffer B (dùng ở SRAM, không dùng trong array)

Bảng 1.2: Hằng dẫn xuất từ P (không phải tham số) (tiếp theo)

Hằng	Giá trị	Công thức, nơi dùng
LP	3	$\log_2 P$
Số PE	64	P·P

Kiểm tra lúc elaboration

- P $\neq$ 8 phải bị chặn lúc elaboration (`$error` trong `generate`)
- `gemm_systolic_array` kiểm tra `L_VPU == gemm_pkg::VPU_LATENCY`; sai thì `$error`. `gemm_pkg::VPU_LATENCY` là nguồn duy nhất của độ trễ VPU; `gemm_accel_top` nối `L_VPU = gemm_pkg::VPU_LATENCY` cho array, không viết số 4 trực tiếp

> **QUYẾT ĐỊNH BASELINE — D-05**
> `MUL_STAGES` = 1, `VPU_LATENCY` = 4, `L_VPU` = 4. `L_VPU` không có giá trị mặc định 0: array luôn tính `o_pe_done` theo độ trễ VPU thật.

1.3 Tín hiệu giao tiếp

Bảng 1.3: Tín hiệu giao tiếp của `gemm_systolic_array`

Tên tín hiệu	Độ rộng	Hướng	Mô tả
Clock, reset			
`clk`	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.
`rst_n`	1	Input	Reset tích cực mức thấp; được kích hoạt bất đồng bộ. Top cấp `rst_n_sync`: cạnh nhả reset đã được đồng bộ với clock.
`i_soft_clr`	1	Input	Xung xóa đồng bộ dài một chu kỳ do Main FSM phát. Xóa trạng thái điều khiển tại cạnh lên; phạm vi xóa dữ liệu được quy định trong mục Reset.
Điều khiển và cấu hình			
`i_pe_start`	1	Input	Xung một chu kỳ bắt đầu COMPUTE cho toàn bộ job. Chỉ phát khi array rảnh; kích thước và cấu hình tính toán phải hợp lệ cùng chu kỳ.
`o_pe_done`	1	Output	Xung một chu kỳ báo COMPUTE đã hoàn tất, bao gồm hậu xử lý và ghi kết quả vào buffer C. Không báo cho lượt tính bị soft clear hủy.
`o_busy`	1	Output	Báo array đang tính. Bằng 1 từ chu kỳ sau khi nhận `i_pe_start` đến hết chu kỳ có `o_pe_done`; về 0 khi soft clear hủy lượt tính.
`i_matrix_dim`	32	Input	Kích thước job, nối từ `o_comp_dim` của Main FSM; nội dung {8'h00, K[7:0], N[7:0], M[7:0]}. Bit [7:0]: M; [15:8]: N; [23:16]: K; bit [31:24] luôn bằng 0 và array bỏ qua. M từ 1 đến 32; N và K là bội của 4, từ 4 đến 32. Phải hợp lệ khi bắt đầu lượt tính và giữ ổn định tới `o_pe_done`.
Cổng đọc buffer A (array là master, không có ready)			

Bảng 1.3: Tín hiệu giao tiếp của `gemm_systolic_array` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`o_a_rd_en`	1	Output	Cho phép đọc một vector A. Địa chỉ hợp lệ trong chu kỳ enable = 1; dữ liệu trả về đúng một chu kỳ sau.
`o_a_rd_row`	5	Output	Hàng đầu i0 của vector A. Là bội của 8; hợp lệ cùng read enable.
`o_a_rd_k`	5	Output	Chỉ số k của cột A cần đọc. Hợp lệ cùng read enable; phải nhỏ hơn K.
`i_a_rd_data`	64	Input	Vector 8 byte INT8 của A (`AB_VEC_WIDTH`). Lane r tại bit [8r +: 8] chứa A[i0+r][k]. Hợp lệ một chu kỳ sau yêu cầu đọc; lane ngoài miền tác vụ mang giá trị bất kỳ.
Cổng đọc buffer B			
`o_b_rd_en`	1	Output	Cho phép đọc một vector B. Địa chỉ hợp lệ trong chu kỳ enable = 1; dữ liệu trả về đúng một chu kỳ sau.
`o_b_rd_k`	5	Output	Chỉ số k của hàng B cần đọc. Hợp lệ cùng read enable; phải nhỏ hơn K.
`o_b_rd_col`	5	Output	Cột đầu j0 của vector B. Là bội của 8; hợp lệ cùng read enable.
`i_b_rd_data`	64	Input	Vector 8 byte INT8 của B (`AB_VEC_WIDTH`). Lane c tại bit [8c +: 8] chứa B[k][j0+c]. Hợp lệ một chu kỳ sau yêu cầu đọc; lane ngoài miền tác vụ mang giá trị bất kỳ.
Luồng hàng kết quả sang VPU (không có ready)			
`o_row_valid`	1	Output	Báo có một đoạn hàng kết quả hợp lệ trong chu kỳ này. Bên nhận phải lấy ngay; giao tiếp không có ready.
`o_row_idx`	5	Output	Chỉ số hàng i của đoạn kết quả. Hợp lệ cùng o_row_valid và phải nhỏ hơn M.
`o_row_col`	5	Output	Cột đầu j0 của đoạn kết quả. Là bội của 8, nhỏ hơn N; hợp lệ cùng o_row_valid.
`o_row_data`	256	Output	Vector 8 giá trị INT32 có dấu (`ROW_VEC_WIDTH`). Lane c tại bit [32c +: 32] chứa C[i][j0+c]; hợp lệ cùng o_row_valid, chỉ dùng lane có mask = 1.
`o_row_mask`	8	Output	Mặt nạ các lane hợp lệ của đoạn hàng: các bit 1 liên tiếp từ bit 0, khác 0. Bit c = 1 khi j0 + c < N; bit c = 0 ngăn ghi phần tử tương ứng. Hợp lệ cùng o_row_valid.

1.3.1 Quy tắc giao tiếp

FSM ↔ Array

TRAO ĐỔI GÌ?

`i_pe_start` bắt đầu một tác vụ; `o_pe_done` báo xong; `o_busy` cho biết đang chạy.

AI KHỞI TẠO?

FSM phát `i_pe_start`; array tự chạy tới cuối và phát `o_pe_done`.

KHI NÀO ĐƯỢC CHẤP NHẬN?

`i_pe_start` chỉ được phát khi `o_busy` = 0. Từ `i_pe_start` tới `o_pe_done`, `i_matrix_dim` phải giữ ổn định.

KHI NÀO SANG BƯỚC TIẾP?

Sau o_pe_done, FSM sang STORE_C; array về rảnh ở chu kỳ kế tiếp.

Array → VPU

TRAO ĐỔI GÌ?
Một đoạn hàng 8 giá trị INT32 kèm hàng, cột đầu và mask (o_row_*).

AI KHỞI TẠO?
Array đẩy; VPU nhận. Không có ready.

KHI NÀO ĐƯỢC CHẤP NHẬN?
VPU nhận ở đúng chu kỳ o_row_valid = 1, kể cả các chu kỳ liền nhau.

KHI NÀO SANG BƯỚC TIẾP?
VPU ghi đoạn hàng vào buffer C đúng L_VPU = 4 chu kỳ sau; array phát o_pe_done L_VPU + 1 = 5 chu kỳ sau đoạn hàng cuối.

Giao tiếp

- Chu kỳ có i_pe_start = 1 gọi là chu kỳ 0. Array chốt M, N, K ở cạnh clock cuối chu kỳ 0. o_busy = 1 từ chu kỳ 1 đến hết chu kỳ có o_pe_done = 1
- Mỗi i_pe_start được chấp nhận có đúng một o_pe_done. i_pe_start khi o_busy = 1 bị bỏ qua (và bị assertion AS-01 báo trong mô phỏng)
- Giá trị M, N, K được giả định hợp lệ (Main FSM đã kiểm tra): $1 \leq M \leq 32$; N, K $\in$ {4, 8, …, 32}. Với giá trị khác, hành vi không xác định; array không kiểm tra lại
- i_matrix_dim[31:24] luôn bằng 0 (Main FSM đã kiểm DIM[31:24] = 0 trước khi phát); array không dùng các bit này, gemm_tile_seq chỉ nhận 24 bit thấp
- o_a_rd_en và o_b_rd_en luôn bằng nhau. Khi bằng 1: o_a_rd_row và o_b_rd_col là bội của 8; o_a_rd_k = o_b_rd_k < K. Khi bằng 0, các port địa chỉ có giá trị tùy ý
- SRAM trả dữ liệu đúng 1 chu kỳ sau chu kỳ có yêu cầu; array không dùng i_a_rd_data, i_b_rd_data ở chu kỳ không có yêu cầu trước đó. Lane ứng với hàng $\geq$ M hoặc cột $\geq$ N được phép mang giá trị bất kỳ, kể cả X trong mô phỏng
- Mọi port o_row_* đều lấy từ thanh ghi. Khi o_row_valid = 1: o_row_idx < M; o_row_mask[c] = 1 khi và chỉ khi $j_0 + c < N$. VPU phải nhận đoạn hàng ở đúng chu kỳ đó
- Không có hai đoạn hàng cho cùng một (i, j_0) trong một tác vụ; mọi (i, j_0) với i < M, j_0 < N đều xuất hiện đúng một lần
- o_pe_done = 1 đúng L_VPU + 1 = 5 chu kỳ sau chu kỳ có o_row_valid = 1 của đoạn hàng cuối cùng. Đoạn hàng cuối được VPU ghi vào buffer C ở cạnh clock cuối chu kỳ D_{last} + 4, tức trước chu kỳ o_pe_done đúng một chu kỳ; do đó tại o_pe_done, mọi kết quả đã nằm trong buffer C

2 Đặc tả thanh ghi

Module không có thanh ghi ánh xạ MMIO. Cấu hình được nhận qua các tín hiệu đầu vào (i_matrix_dim, i_pe_start); các thanh ghi MMIO của hệ thống nằm trong gemm_register_file, xem đặc tả hệ thống baseline. Trạng thái nội bộ, điều kiện chốt và reset được mô tả bên dưới.

3 Mô tả chức năng

3.1 Tổ chức chức năng

Array gồm 4 khối: **bộ tuần tự tile** phát yêu cầu đọc và cờ điều khiển; **khối làm lệch** trễ lane r của A r chu kỳ và lane c của B c chu kỳ; **lưới PE** 8×8; **khối xả** lấy kết quả từng hàng khi hàng đó vừa xong và xuất sang VPU. Mọi khối chạy theo nhịp cố định, không có bắt tay bên trong.

Bảng 3.1: Module con

Module	Instance	Vai trò
`gemm_tile_seq`	`u_seq`	Chốt M, N, K; đếm tile và k; phát yêu cầu đọc A, B; sinh cờ `row_vld`, `first`, `last`; đẩy metadata tile cho khối xả
`gemm_skew`	`u_skew`	Trễ cờ điều khiển 1 chu kỳ cho khớp độ trễ SRAM; làm lệch A (cùng cờ) và B
`gemm_pe_grid`	`u_grid`	8×8 instance `gemm_pe` nối thành lưới
`gemm_pe`	`u_pe[r][c]`	Một phần tử xử lý: nhân-cộng, giữ `acc`, `res`
`gemm_drain`	`u_drain`	FIFO metadata; chọn hàng `res` đang sẵn sàng; thanh ghi đầu ra `o_row_*`; cờ `final`
(logic trong top)	—	`o_busy`; đường trễ `L_VPU` + 1 = 5 chu kỳ từ `final` tới `o_pe_done`

3.1.1 Nạp dữ liệu, lịch MAC và xả kết quả

> **QUYẾT ĐỊNH BASELINE — D-30**
> Lịch dưới đây là quy định, RTL phải khớp từng chu kỳ. Nguồn: cả hai tài liệu nguồn nhất quán; baseline gom lại thành một lịch duy nhất.

- **Chiều dữ liệu**: A chảy theo hàng sang đông, B chảy theo cột xuống nam, mỗi hop một chu kỳ. PE(r, c) nhận A từ PE(r, c−1) và B từ PE(r−1, c)
- **Làm lệch**: `gemm_skew` cho lane A hàng r đi qua đúng r thanh ghi và lane B cột c đi qua đúng c thanh ghi. Với P = 8 là 0 + 1 + … + 7 = 28 tầng mỗi phía: mỗi tầng A mang 11 bit (8 bit dữ liệu + 3 bit cờ `vld`, `first`, `last`), mỗi tầng B mang 8 bit. Hệ quả: phần tử k của tile n tới PE(r, c) ở chu kỳ $S_n + 1 + k + r + c$
- **Thứ tự tile và khe**: t_j trong, t_i ngoài; $i_0 = 8t_i$, $j_0 = 8t_j$. Mỗi tile chiếm $I = \max(K, 8)$ khe liên tiếp, `cnt` = 0 … I−1. Khe `cnt` < K phát đọc k = `cnt` với `first` = (`cnt` == 0), `last` = (`cnt` == K−1), `row_vld[r]` = ($i_0 + r$ < M); khe K ≤ `cnt` < I là bọt (`o_rd_en` = 0)
- **MAC**: một chu kỳ mỗi bước. `i_first` nạp tích mới (`acc` ← a·b); các bước sau `acc` ← `acc` + a·b; chỉ cập nhật khi `i_vld` = 1. `i_last` chốt `res` ← tổng cuối
- **Xả**: `trig[r]` = `i_east_vld[r]` ∧ `i_east_last[r]` là one-hot, tối đa một hàng mỗi chu kỳ. Khối xả giữ FIFO metadata sâu 2 (`i0`, `j0`, `mask`, `rows`, `last_tile`), chọn hàng bằng mux 8:1 one-hot trên 8×32 bit và đẩy một đoạn hàng 256 bit mỗi chu kỳ
- **Báo xong**: `o_pe_done` = `o_final` trễ `L_VPU` + 1 = 5 chu kỳ, bảo đảm VPU đã ghi xong đoạn hàng cuối vào buffer C

3.1.2 M, N không chia hết cho 8 và K

> **QUYẾT ĐỊNH BASELINE — D-26**
> Phần cứng xử lý M, N không chia hết cho 8 mà không cần phần mềm đệm. Phần mềm chỉ phải đệm N và K lên bội của 4 (điều kiện của descriptor hợp lệ); M tùy ý 1 … 32.

- $T_M = \lceil M/8 \rceil$, $T_N = \lceil N/8 \rceil$, tính bằng dịch (không chia). Tile biên có số hàng hợp lệ rows = $\min(8, M - i_0)$
- **Hàng thừa**: `row_vld[r]` = ($i_0 + r$ < M). Hàng PE r có `row_vld[r]` = 0 mang `vld` = 0 nên `acc` không cập nhật và `trig[r]` không bao giờ bật: hàng thừa không được xả

- **Cột thừa**: `o_row_mask` (bit c = 1 khi $j_0 + c <$ N, các bit 1 liên tiếp từ bit 0, khác 0). PE của cột thừa vẫn tính trên dữ liệu lane mà SRAM trả về (giá trị bất kỳ, kể cả X trong mô phỏng); kết quả đó không bao giờ được ghi vì `o_row_mask` → `i_c_wr_mask` của VPU → byte/word enable của bank C
- **K**: không cần xử lý riêng. `gemm_tile_seq` phát đúng K bước (`cnt` < K) rồi chèn bọt tới $I = \max(K, 8)$. K < 8 cho I = 8 (có bọt); K $\geq$ 8 cho I = K. K không bội 8 vẫn hợp lệ, chỉ cần bội 4
- Ví dụ M = 20, N = 12: T_M = 3, T_N = 2, N_T = 6. Tile cuối có i_0 = 16, rows = 4, `row_vld` = 8'b0000_1111; các tile có t_j = 1 có j_0 = 8 và `o_row_mask` = 8'b0000_1111

3.2 Module con

3.2.1 `gemm_pe`

Bảng 3.2: Tín hiệu giao tiếp của `gemm_pe`

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`clk`	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.
`rst_n`	1	Input	Reset tích cực mức thấp; được kích hoạt bất đồng bộ. Cạnh nhả reset được đồng bộ với clock tại top trước khi đưa tới các module.
`i_clr`	1	Input	Xóa đồng bộ các trạng thái điều khiển tại cạnh lên. Được nối với tín hiệu soft clear của module cha.
`i_a`	8	Input	Phần tử A kiểu INT8 có dấu, đến từ PE bên trái hoặc khối căn chỉnh. Tham gia phép nhân khi i_vld = 1.
`i_vld`	1	Input	Báo cặp dữ liệu A/B hợp lệ trong chu kỳ này. Khi bằng 1, PE cập nhật tích lũy tại cạnh lên.
`i_first`	1	Input	Đánh dấu phần tử k = 0 của tile khi dữ liệu hợp lệ. PE nạp tích mới thay vì cộng vào tích lũy cũ.
`i_last`	1	Input	Đánh dấu phần tử k = K − 1 của tile khi dữ liệu hợp lệ. PE chốt tổng cuối vào thanh ghi kết quả.
`i_b`	8	Input	Phần tử B kiểu INT8 có dấu, đến từ PE phía trên hoặc khối căn chỉnh. Tham gia phép nhân khi i_vld = 1.
`o_a`	8	Output	Bản trễ một chu kỳ của i_a, truyền sang PE bên phải.
`o_vld`	1	Output	Bản trễ một chu kỳ của i_vld, truyền cùng dữ liệu A sang PE bên phải.
`o_first`	1	Output	Bản trễ một chu kỳ của i_first, truyền sang PE bên phải để đánh dấu đầu tile.
`o_last`	1	Output	Bản trễ một chu kỳ của i_last, truyền sang PE bên phải để đánh dấu cuối tile.
`o_b`	8	Output	Bản trễ một chu kỳ của i_b, truyền xuống PE bên dưới.
`o_res`	32	Output	Tổng INT32 của tile gần nhất đã hoàn tất. Lấy từ thanh ghi res và giữ nguyên tới lần chốt kết quả tile tiếp theo.

$$sum = \big(i_first\ ?\ 0 : acc\big) + \mathrm{sext}_{32}\big(i_a \times i_b\big) \qquad \begin{array}{ll} \text{nếu } i_vld: & acc \leftarrow sum \\ \text{nếu } i_vld \wedge i_last: & res \leftarrow sum \end{array}$$

gemm_pe

- Phép nhân là INT8 $\times$ INT8 có dấu cho tích 16 bit có dấu, mở rộng dấu lên 32 bit; phép cộng 32 bit wrap mod 2^{32} (không saturate). Trên miền hợp lệ $|a \cdot b| \leq 2^{14}$ và K $\leq$ 32 nên $|C| \leq 2^{19}$: không bao giờ tràn, quy tắc wrap chỉ để định nghĩa hành vi hình thức. Toàn bộ nhân-cộng nằm trong một chu kỳ (tổ hợp từ `i_a`, `i_b`, `acc` tới thanh ghi)
- `acc` và `res` chỉ thay đổi khi `i_vld` = 1 (và `i_last` = 1 với `res`). `i_first` = 1 nạp tích mới, không cộng với `acc` cũ; không có chu kỳ xóa
- `o_a`, `o_b`, `o_vld`, `o_first`, `o_last` cập nhật mọi chu kỳ (không có điều kiện chặn)
- `rst_n` = 0 hoặc `i_clr` = 1: `o_vld`, `o_first`, `o_last` về 0. `acc`, `res`, `o_a`, `o_b` không có giá trị reset
- `o_res` nối thẳng từ thanh ghi `res`. Thanh ghi mỗi PE: 83 bit (`acc` 32 + `res` 32 + a 8 + b 8 + 3 cờ)

```
// Tham khảo (SystemVerilog)
logic signed [15:0] prod;
logic signed [31:0] sum;
assign prod = $signed(i_a) * $signed(i_b);
assign sum  = (i_first ? 32'sd0 : acc) + 32'(prod);
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n)      {o_vld, o_first, o_last} <= '0;
  else if (i_clr)  {o_vld, o_first, o_last} <= '0;
  else             {o_vld, o_first, o_last} <= {i_vld, i_first, i_last};
always_ff @(posedge clk) begin
  o_a <= i_a;  o_b <= i_b;
  if (i_vld)           acc <= sum;
  if (i_vld && i_last) res <= sum;
end
```

3.2.2 `gemm_tile_seq`

Bảng 3.3: Tín hiệu giao tiếp của `gemm_tile_seq`

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`clk`	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.
`rst_n`	1	Input	Reset tích cực mức thấp; được kích hoạt bất đồng bộ. Cạnh nhả reset được đồng bộ với clock tại top trước khi đưa tới các module.
`i_clr`	1	Input	Xóa đồng bộ các trạng thái điều khiển tại cạnh lên. Được nối với tín hiệu soft clear của module cha.
`i_start`	1	Input	Xung một chu kỳ bắt đầu duyệt tile khi bộ tuần tự đang IDLE. Chốt kích thước ma trận tại cạnh lên (nối từ `i_pe_start`).
`i_matrix_dim`	24	Input	Kích thước job: bit [7:0]: M; [15:8]: N; [23:16]: K (24 bit thấp của `i_matrix_dim` của array). M từ 1 đến 32; N và K là bội của 4, từ 4 đến 32. Phải hợp lệ khi bắt đầu lượt tính.
`o_rd_en`	1	Output	Cho phép đọc đồng thời vector A và B trong chu kỳ này. Được nối tới cả hai read enable của SRAM.
`o_rd_i0`	5	Output	Hàng đầu i0 của tile đang đọc; là bội của 8. Hợp lệ khi o_rd_en = 1.
`o_rd_j0`	5	Output	Cột đầu j0 của tile đang đọc; là bội của 8. Hợp lệ khi o_rd_en = 1.

Bảng 3.3: Tín hiệu giao tiếp của `gemm_tile_seq` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`o_rd_k`	5	Output	Chỉ số k chung cho cột A và hàng B đang đọc, từ 0 đến K − 1. Hợp lệ khi o_rd_en = 1.
`o_row_vld`	8	Output	Mặt nạ hàng hợp lệ. Bit r = 1 khi i0 + r < M; chỉ có nghĩa cùng o_rd_en = 1.
`o_first`	1	Output	Đánh dấu k = 0, phần tử đầu tile. Chỉ có nghĩa khi o_rd_en = 1.
`o_last`	1	Output	Đánh dấu k = K − 1, phần tử cuối tile. Chỉ có nghĩa khi o_rd_en = 1.
`o_meta_push`	1	Output	Xung đẩy metadata của tile vào khối xả. Bằng 1 khi o_rd_en = 1 và k = K − 1.
`o_meta_i0`	5	Output	Hàng đầu i0 của tile. Metadata được lấy mẫu khi o_meta_push = 1.
`o_meta_j0`	5	Output	Cột đầu j0 của tile. Metadata được lấy mẫu khi o_meta_push = 1.
`o_meta_mask`	8	Output	Mặt nạ cột hợp lệ của tile. Bit c = 1 khi j0 + c < N; lấy mẫu khi o_meta_push = 1.
`o_meta_rows`	4	Output	Số hàng hợp lệ của tile, bằng min(8, M − i0), từ 1 đến 8 ($\lceil \log_2(P+1) \rceil$ = 4 bit). Lấy mẫu cùng o_meta_push.
`o_meta_last_tile`	1	Output	Đánh dấu metadata thuộc tile cuối của tác vụ. Lấy mẫu cùng o_meta_push.

`gemm_tile_seq`

- Hai trạng thái: IDLE (trạng thái reset) và ISSUE. `i_start` ở IDLE: chốt M, N, K; tính T_M, T_N, $N_T = T_M T_N$ và $I = \max(K, 8)$; đặt tile n = 0, (t_i, t_j) = (0, 0), bộ đếm khe `cnt` = 0; sang ISSUE
- Trong ISSUE, mỗi chu kỳ là một "khe" `cnt` = 0 … I−1 của tile n. Khe `cnt` < K phát yêu cầu đọc k = `cnt`, `first` = (`cnt` == 0), `last` = (`cnt` == K−1); khe `cnt` ≥ K là bọt (`o_rd_en` = 0). Sau khe I−1: sang tile kế tiếp theo thứ tự t_j chạy trong, t_i chạy ngoài; sau tile $N_T - 1$: về IDLE
- Mọi output là hàm của thanh ghi trạng thái (không phụ thuộc tổ hợp vào input trong cùng chu kỳ). $i_0 = 8t_i$, $j_0 = 8t_j$
- Hệ quả: yêu cầu đọc k của tile n nằm ở chu kỳ $S_n + k$, với $S_n = 1 + n \cdot I$ (chu kỳ 0 là chu kỳ có `i_pe_start`; khe đầu tiên ở chu kỳ 1)

3.2.3 `gemm_skew`

Bảng 3.4: Tín hiệu giao tiếp của `gemm_skew`

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`clk`	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.
`rst_n`	1	Input	Reset tích cực mức thấp; được kích hoạt bất đồng bộ. Cạnh nhả reset được đồng bộ với clock tại top trước khi đưa tới các module.
`i_clr`	1	Input	Xóa đồng bộ các trạng thái điều khiển tại cạnh lên. Được nối với tín hiệu soft clear của module cha.

Bảng 3.4: Tín hiệu giao tiếp của `gemm_skew` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`i_rd_en`	1	Input	Enable đọc do bộ tuần tự phát. Cùng chu kỳ với yêu cầu SRAM; dùng để căn cờ hợp lệ với dữ liệu về ở chu kỳ sau.
`i_row_vld`	8	Input	Mặt nạ hàng hợp lệ do bộ tuần tự phát cùng yêu cầu đọc. Bit r = 0 chặn phép tích lũy của hàng PE tương ứng.
`i_first`	1	Input	Cờ đầu tile do bộ tuần tự phát cùng yêu cầu đọc. Được căn trễ với dữ liệu A trước khi đưa vào PE.
`i_last`	1	Input	Cờ cuối tile do bộ tuần tự phát cùng yêu cầu đọc. Được căn trễ với dữ liệu A trước khi đưa vào PE.
`i_a_data`	64	Input	Vector A gồm 8 byte từ SRAM, hợp lệ một chu kỳ sau yêu cầu đọc. Lane r được làm trễ thêm r chu kỳ trước khi vào hàng PE r.
`i_b_data`	64	Input	Vector B gồm 8 byte từ SRAM, hợp lệ một chu kỳ sau yêu cầu đọc. Lane c được làm trễ thêm c chu kỳ trước khi vào cột PE c.
`o_west_a`	64	Output	Vector A đã căn chỉnh. Lane r cấp dữ liệu cho PE(r, 0), trễ thêm r chu kỳ so với dữ liệu SRAM.
`o_west_vld`	8	Output	Cờ hợp lệ đã căn chỉnh cho các hàng PE. Bit r đi cùng lane A của PE(r, 0).
`o_west_first`	8	Output	Cờ đầu tile đã căn chỉnh. Bit r đi cùng lane A của PE(r, 0).
`o_west_last`	8	Output	Cờ cuối tile đã căn chỉnh. Bit r đi cùng lane A của PE(r, 0).
`o_north_b`	64	Output	Vector B đã căn chỉnh. Lane c cấp dữ liệu cho PE(0, c), trễ thêm c chu kỳ so với dữ liệu SRAM.

`gemm_skew`

- Tầng căn chỉnh: một thanh ghi chốt {`i_rd_en` ∧ `i_row_vld[r]`, `i_first`, `i_last`} cho mỗi hàng r (8 × 3 = 24 bit), cho cờ của mỗi hàng ở chu kỳ sau yêu cầu, cùng lúc với dữ liệu SRAM. Thanh ghi này reset/clear về 0
- Lane r của A cùng ba cờ của hàng r đi qua đúng r thanh ghi nối tiếp; lane 0 không qua thanh ghi nào. Thanh ghi cờ reset/clear về 0
- Lane c của B đi qua đúng c thanh ghi nối tiếp; lane 0 không qua thanh ghi nào
- Tổng số tầng: 28 cho A (mỗi tầng 8 + 3 = 11 bit, 308 bit) và 28 cho B (mỗi tầng 8 bit, 224 bit), cộng 24 bit tầng căn chỉnh

3.2.4 `gemm_pe_grid`

> **QUYẾT ĐỊNH BASELINE**
> Nguồn spec_1 mô tả lưới bằng văn xuôi, không có bảng cổng. Baseline đặt tên cổng dưới đây (cùng hậu tố với `gemm_skew` và `gemm_drain`) để bảng cổng đầy đủ như các module con khác; nội dung nối dây không đổi so với nguồn.

Bảng 3.5: Tín hiệu giao tiếp của `gemm_pe_grid`

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`clk`	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.
`rst_n`	1	Input	Reset tích cực mức thấp; được kích hoạt bất đồng bộ. Cạnh nhả reset được đồng bộ với clock tại top trước khi đưa tới các module.
`i_clr`	1	Input	Xóa đồng bộ các trạng thái điều khiển tại cạnh lên; nối tới `i_clr` của mọi PE.
`i_west_a`	64	Input	Vector A đã căn chỉnh từ `o_west_a`; lane r vào `i_a` của PE(r, 0).
`i_west_vld`	8	Input	Cờ hợp lệ từ `o_west_vld`; bit r vào `i_vld` của PE(r, 0).
`i_west_first`	8	Input	Cờ đầu tile từ `o_west_first`; bit r vào `i_first` của PE(r, 0).
`i_west_last`	8	Input	Cờ cuối tile từ `o_west_last`; bit r vào `i_last` của PE(r, 0).
`i_north_b`	64	Input	Vector B đã căn chỉnh từ `o_north_b`; lane c vào `i_b` của PE(0, c).
`o_res_bus`	2048	Output	`o_res` của mọi PE; PE(r, c) tại bit [(r·8+c)·32 +: 32].
`o_east_vld`	8	Output	Bit r = `o_vld` của PE(r, 7).
`o_east_last`	8	Output	Bit r = `o_last` của PE(r, 7).

`gemm_pe_grid`

- PE(r, c), $0 \le r, c < 8$. Đầu vào A và cờ của PE(r, 0) lấy từ `i_west_*` lane r; của PE(r, c > 0) lấy từ `o_a`, `o_vld`, `o_first`, `o_last` của PE(r, c−1)
- `i_b` của PE(0, c) lấy từ `i_north_b` lane c; của PE(r > 0, c) lấy từ `o_b` của PE(r−1, c)
- Output của lưới: `o_res` của mọi PE, xếp `res_bus[(r·8+c)·32+:32]`; `o_vld` và `o_last` của PE(r, 7) cho mỗi r (`east_vld[r]`, `east_last[r]`)
- `o_a` của cột cuối và `o_b` của hàng cuối không nối đi đâu (công cụ tổng hợp tự bỏ)

3.2.5 `gemm_drain`

Bảng 3.6: Tín hiệu giao tiếp của `gemm_drain`

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`clk`	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.
`rst_n`	1	Input	Reset tích cực mức thấp; được kích hoạt bất đồng bộ. Cạnh nhả reset được đồng bộ với clock tại top trước khi đưa tới các module.
`i_clr`	1	Input	Xóa đồng bộ các trạng thái điều khiển tại cạnh lên. Được nối với tín hiệu soft clear của module cha.
`i_res_bus`	2048	Input	Bus chứa kết quả INT32 của mọi PE ($32 \cdot P^2$ bit). Phần tử PE(r, c) nằm tại bit [(r·8+c)·32 +: 32].
`i_east_vld`	8	Input	Vector cờ hợp lệ tại cột PE cuối. Bit r lấy từ o_vld của PE(r, 7).

Bảng 3.6: Tín hiệu giao tiếp của `gemm_drain` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`i_east_last`	8	Input	Vector cờ cuối tile tại cột PE cuối. Bit r lấy từ o_last của PE(r, 7); kết hợp với east valid để kích hoạt xả hàng r.
`i_meta_push`	1	Input	Xung nhận metadata của tile vào FIFO. Các trường metadata còn lại được lấy mẫu cùng chu kỳ.
`i_meta_i0`	5	Input	Hàng đầu của tile được xả. Hợp lệ khi i_meta_push = 1.
`i_meta_j0`	5	Input	Cột đầu của tile được xả; là bội của 8. Hợp lệ khi i_meta_push = 1.
`i_meta_mask`	8	Input	Mặt nạ cột hợp lệ; bit c = 1 khi cột j0 + c thuộc ma trận kết quả. Lấy mẫu cùng i_meta_push.
`i_meta_rows`	4	Input	Số hàng hợp lệ cần xả, từ 1 đến 8. Lấy mẫu cùng i_meta_push.
`i_meta_last_tile`	1	Input	Đánh dấu metadata của tile cuối tác vụ. Lấy mẫu cùng i_meta_push.
`o_row_valid`	1	Output	Báo đoạn hàng đã đăng ký ở đầu ra và hợp lệ trong chu kỳ này. Bên nhận phải nhận ngay; không có ready.
`o_row_idx`	5	Output	Chỉ số hàng của đoạn kết quả. Hợp lệ cùng o_row_valid.
`o_row_col`	5	Output	Cột đầu của đoạn kết quả, là bội của 8. Hợp lệ cùng o_row_valid.
`o_row_data`	256	Output	8 phần tử INT32 của đoạn hàng. Lane c ở bit [32c +: 32]; chỉ dùng các lane có mask bằng 1.
`o_row_mask`	8	Output	Mặt nạ lane được phép ghi. Bit c = 1 cho biết phần tử c thuộc miền hợp lệ; đi cùng o_row_valid.
`o_final`	1	Output	Xung một chu kỳ báo đoạn hàng cuối của tile cuối đang được xuất. Cùng chu kỳ với o_row_valid của đoạn hàng đó.

`gemm_drain`

- Tín hiệu kích hoạt của hàng r: `trig[r]=i_east_vld[r]∧i_east_last[r]`. Nó bằng 1 đúng ở chu kỳ đầu tiên mà `res` của cả hàng r đã chứa kết quả của tile vừa xong. Tại mỗi chu kỳ có tối đa một bit `trig` bằng 1 (one-hot, hoặc toàn 0 khi không có hàng nào xong)
- FIFO metadata sâu 2 mục (`i0`, `j0`, `mask`, `rows`, `last_tile`): đẩy khi `i_meta_push`, lấy ra khi `trig[0]` = 1. Mục vừa lấy ra trở thành "tile đang xả" và được giữ tới lần `trig[0]` kế tiếp
- Khi `trig[r]` = 1 (với r = 0 dùng mục vừa lấy ra): ở cạnh clock cuối chu kỳ đó, thanh ghi đầu ra nạp `o_row_valid` = 1, `o_row_idx` = $i_0 + r$, `o_row_col` = j_0, `o_row_data` = `res` của hàng r, `o_row_mask` = mask của tile, `o_final` = (`last_tile` ∧ r = `rows` − 1). Không có `trig`: `o_row_valid` = `o_final` = 0
- Hàng không hợp lệ ($i_0 + r \geq M$) có `vld` = 0 trong mọi PE của nó, nên `trig[r]` không bao giờ bật cho hàng đó: không cần lọc thêm
- Mux chọn hàng là 8:1 trên 8×32 bit (256 bit), điều khiển bởi `trig` (one-hot)

3.2.6 `o_busy` và `o_pe_done`

Logic trong top

- `o_pe_done` = `o_final` trễ đúng `L_VPU` + 1 = 5 chu kỳ (đường trễ 5 thanh ghi, reset/clear về 0)
- `o_busy`: đặt 1 ở cạnh clock cuối chu kỳ có `i_pe_start` (khi đang rảnh), về 0 ở cạnh clock cuối chu kỳ có `o_pe_done`. `i_pe_start` khi `o_busy` = 1 bị bỏ qua
- Khi có `i_soft_clr`, `o_busy` về 0 (mục Reset)

3.3 Timing

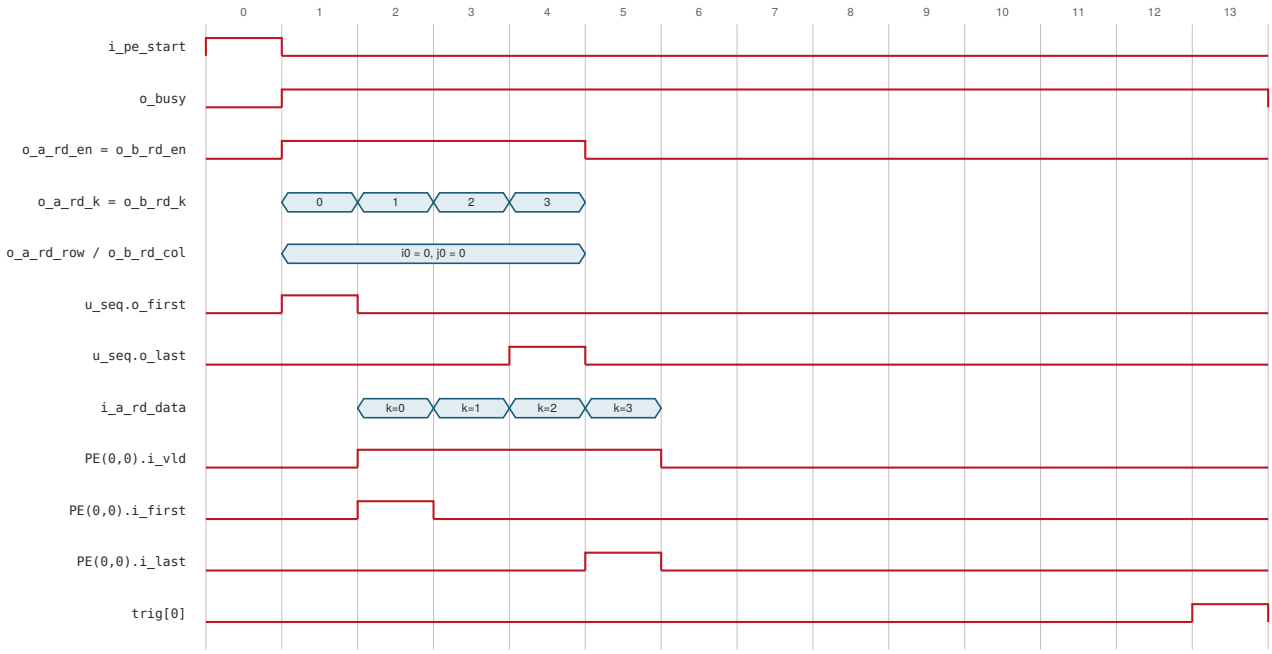
Mọi sự kiện có thời điểm cố định, tính được từ M, N, K và `L_VPU` = 4. Các công thức dưới đây là quy định: RTL phải khớp đúng từng chu kỳ (kiểm bằng `gemm_systolic_array_model.py`; mô hình nằm ở `gemm-accel/model/`, đã chạy lại với baseline ngày 12/11/2026: 330 test, 0 lỗi; (M, N, K) = (32, 32, 32) cho 534 chu kỳ COMPUTE, khớp bảng 3.8). Cột ví dụ lấy K = 4, P = 8, tile n = 0, hàng r = 0.

Bảng 3.7: Thời điểm các sự kiện (chu kỳ 0 = chu kỳ có `i_pe_start`)

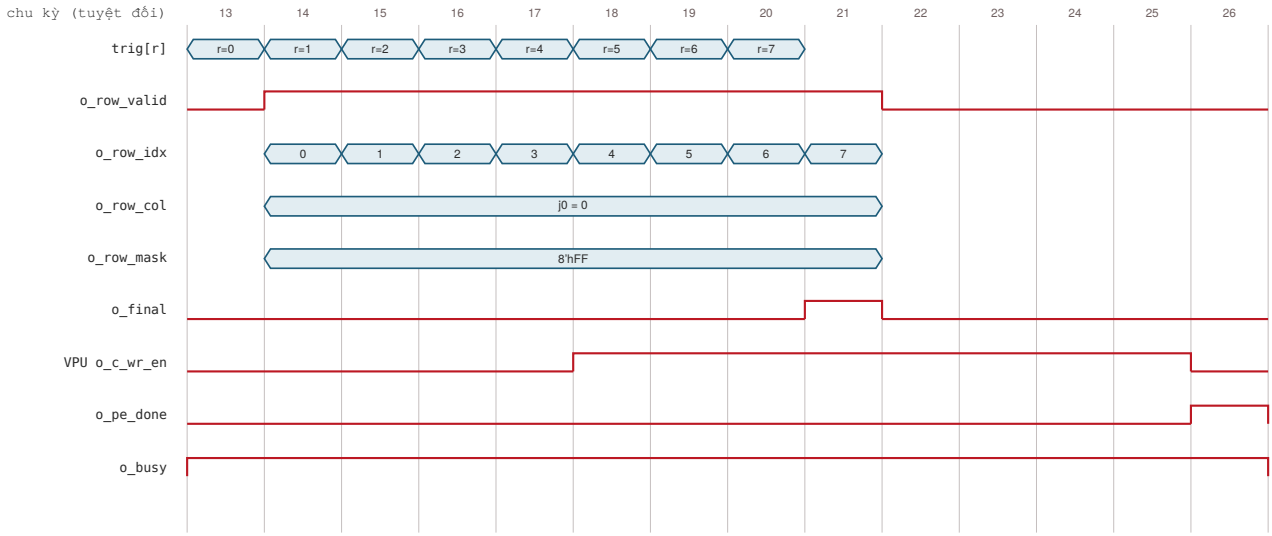
Sự kiện	Chu kỳ	Ví dụ (n = 0, K = 4)
Khoảng cách giữa hai tile	$I = \max(K, 8)$	8
Yêu cầu đọc k của tile n	$S_n + k$, với $S_n = 1 + nI$	1 … 4
Dữ liệu SRAM của yêu cầu đó	$S_n + k + 1$	2 … 5
PE(r, c) nhận phần tử k của tile n	$S_n + 1 + k + r + c$	PE(0, 0): 2 … 5
`res` của PE(r, c) chứa tile n từ	$S_n + 1 + K + r + c$	PE(0, 0): 6
`trig[r]` của tile n	$S_n + K + P + r$	13 (r = 0)
`o_row_valid` của hàng r, tile n	$D_n(r) = S_n + K + P + 1 + r$	14 (r = 0)
`o_pe_done`	$D_{last} + L_{VPU} + 1 = (N_T - 1)I + K + P + r_{last} + L_{VPU} + 3$	26 (M = N = 8)

Trong đó $N_T = \lceil M/8 \rceil \lceil N/8 \rceil$ và $r_{last} = \min(8, M - (T_M - 1) \cdot 8) - 1$ là **chỉ số** của hàng hợp lệ cuối của tile cuối, tức (số hàng hợp lệ của tile cuối) − 1.

> **QUYẾT ĐỊNH BASELINE — D-06**
> r_{last} là chỉ số, không phải số lượng. Công thức COMPUTE của toàn hệ thống: COMPUTE_CYCLES $= (N_T - 1) \cdot \max(K, P) + K + P + r_{last} + \text{LAT} + 3$ với LAT = `L_VPU` = 4.
> Kiểm chứng M = N = K = 32: $T_M = T_N = 4$, $N_T = 16$, $I = \max(32, 8) = 32$, $r_{last} = 7$, LAT = 4 → 15·32 + 32 + 8 + 7 + 4 + 3 = **534 chu kỳ** = 5,34 µs ở 100 MHz.



Hình 3.1: Một tile, K = 4, P = 8, M = N = 8 (S_0 = 1, I = 8): từ `i_pe_start` (chu kỳ 0) tới hàng đầu sẵn sàng xả. Đọc k ở chu kỳ 1 … 4; dữ liệu SRAM về ở 2 … 5; PE(0, 0) nhận phần tử k ở $S_0 + 1 + k$ = 2 … 5 cùng cờ `first` (chu kỳ 2) và `last` (chu kỳ 5); `trig[0]` = $S_0 + K + P$ = 13



Hình 3.2: Xả tile của Hình 3.1 (trục thời gian dịch 13 chu kỳ: ô 0 của lưới là chu kỳ tuyệt đối 13). Hàng r xả ở $D_0(r) = S_0 + K + P + 1 + r$ = 14 + r (14 … 21), một hàng mỗi chu kỳ; `o_final` ở chu kỳ 21 cùng hàng 7; VPU ghi buffer C ở chu kỳ D + 4 = 18 … 25; `o_pe_done` ở 21 + 5 = 26 = 0 + 4 + 8 + 7 + 4 + 3. `o_busy` về 0 từ chu kỳ 27

VÌ SAO I = MAX(K, P) LÀ ĐỦ

- $I \geq K$: mỗi tile cần K lần đọc liên tiếp trên một cổng.
- $I \geq P$: `res` của PE(r, 0) bị tile n+1 ghi đè ở cạnh clock cuối chu kỳ $S_n + I + K + r$, không sớm hơn chu kỳ $S_n + K + P + r$ mà khối xả đọc hàng r của tile n. Thanh ghi đọc trước khi bị ghi trong cùng cạnh clock, nên giá trị xả là của tile n.
- $I \geq P$ cũng làm 8 đoạn hàng của một tile và của tile sau không bao giờ trùng chu kỳ, nên cổng xả chỉ cần một đoạn hàng mỗi chu kỳ và FIFO metadata chỉ cần 2 mục.

Bảng 3.8: Số chu kỳ từ `i_pe_start` tới `o_pe_done` (P = 8, `L_VPU` = 4)

(M, N, K)	Số chu kỳ	Tính
(32, 32, 32)	534	$15 \cdot 32 + 32 + 8 + 7 + 4 + 3$
(20, 12, 8)	66	$N_T = 6$, $r_{last} = 3$: $5 \cdot 8 + 8 + 8 + 3 + 4 + 3$
(8, 8, 4) (Hình 3.1, 3.2)	26	$0 + 4 + 8 + 7 + 4 + 3$
(2, 4, 4)	20	$r_{last} = 1$: $0 + 4 + 8 + 1 + 4 + 3$
(1, 4, 4)	19	$r_{last} = 0$: $0 + 4 + 8 + 0 + 4 + 3$

Thông lượng: một đoạn hàng mỗi chu kỳ ở cổng xả; một tile mỗi I chu kỳ. Đỉnh tính toán: 64 MAC mỗi chu kỳ.

Lịch kiểm chứng Testbench kiểm theo lịch tuyệt đối: đọc k của tile n ở chu kỳ $S_n + k$ với $S_n = 1 + n \cdot I$; hàng r của tile n xả ở $D_n(r) = S_n + K + P + 1 + r$; `o_pe_done` ở $D_{N_T-1}(r_{last}) + 5$.

3.4 Reset và soft clear

> **QUYẾT ĐỊNH BASELINE — D-21**
> Flip-flop trạng thái và bit hợp lệ dùng `always_ff@(posedgeclkornegedgerst_n)` (`rst_n` ở đây là `rst_n_sync` do top cấp, xuống thấp bất đồng bộ). Flip-flop chỉ chứa dữ liệu dùng `always_ff@(posedgeclk)` và không có giá trị reset. Không có latch.

Reset

- `rst_n` = 0 (bất đồng bộ): `gemm_tile_seq` về IDLE; mọi thanh ghi cờ (`vld`, `first`, `last` trong `gemm_skew` và `gemm_pe`), FIFO metadata, `o_row_valid`, `o_final`, đường trễ `o_pe_done`, `o_busy` về 0
- `i_soft_clr` = 1 (đồng bộ): tác động như `rst_n` = 0 lên cùng tập thanh ghi trên, ở cạnh clock cuối chu kỳ đó. Từ chu kỳ kế tiếp array rảnh; không có `o_pe_done` cho lượt bị hủy, không có `o_row_valid` nào của lượt đó
- Thanh ghi dữ liệu (`acc`, `res`, `o_a`, `o_b` trong PE và tầng làm lệch, dữ liệu đầu ra `o_row_data`) không có giá trị reset; mọi đường dùng chúng đều bị chặn bởi cờ hợp lệ đã được reset
- `i_soft_clr` và `i_pe_start` không bao giờ cùng chu kỳ (FSM bảo đảm); nếu có, `i_soft_clr` thắng

3.5 Quy tắc hiện thực

Hiện thực

- SystemVerilog tổng hợp được; `always_ff@(posedgeclkornegedgerst_n)` cho thanh ghi trạng thái và bit hợp lệ, `always_ff@(posedgeclk)` cho thanh ghi dữ liệu thuần (`acc`, `res`, `o_a`, `o_b`, dữ liệu tầng làm lệch) không có giá trị reset; không có latch
- Mọi phép nhân, cộng dùng kiểu `signed` rõ ràng (`$signed`); không dùng phép chia hay modulo: $\lceil M/8 \rceil$ tính bằng dịch
- Mỗi module con trong file riêng, cùng tên module; lưới PE và các tầng làm lệch dựng bằng `generate`
- Không có đường tổ hợp từ input tới output của `gemm_systolic_array`: các port `o_*` đều lấy từ thanh ghi
- Array không có đầu ra lỗi; giá trị M, N, K ngoài miền cho hành vi không xác định (Main FSM đã loại trước khi phát `i_pe_start`)

Assertion bắt buộc trong RTL (SVA, chỉ trong mô phỏng)

- **AS-01**: `i_pe_start` → `!o_busy`
- **AS-02**: `o_a_rd_en==o_b_rd_en`
- **AS-03**: `$onehot0(trig)` trong `gemm_drain`

- **AS-04**: FIFO metadata không bao giờ đẩy khi đầy, không bao giờ lấy khi rỗng
- **AS-05**: `o_busy` → `$stable(i_matrix_dim)`
- **AS-06**: `o_row_valid` → (`o_row_idx` < M) ∧ (`o_row_col` < N)
- **AS-07**: mỗi `i_pe_start` có đúng một `o_pe_done` trước `i_pe_start` kế tiếp
- **AS-08**: `o_pe_done` bằng `o_final` trễ đúng `L_VPU` + 1 = 5 chu kỳ (khi không có `i_soft_clr` trong khoảng đó)
- **AS-09**: `i_pe_start` → `i_matrix_dim[31:24] == 0`
- Kiểm chéo ở `gemm_accel_top`: `u_array.o_pe_done` → không còn bit hợp lệ nào trong pipeline VPU (AS-07 của `gemm_vpu`)

Bảng 3.9: Tài nguyên ước tính (P = 8)

Hạng mục	Số lượng
Bộ nhân 8 × 8 có dấu	64
Bộ cộng 32 bit	64
Thanh ghi trong lưới PE (83 bit mỗi PE)	5 312 bit
Thanh ghi làm lệch (A 11 bit, B 8 bit mỗi tầng)	28 + 28 tầng: 532 bit
Thanh ghi tầng căn chỉnh cờ (8 hàng × 3 bit)	24 bit
FIFO metadata	2 mục
Mux xả	8:1 × 256 bit